MASTER OF SCIENCE
IN ENGINEERING

Recurrent Neural Networks - 2

Part 2: Seq2Seq - Attention

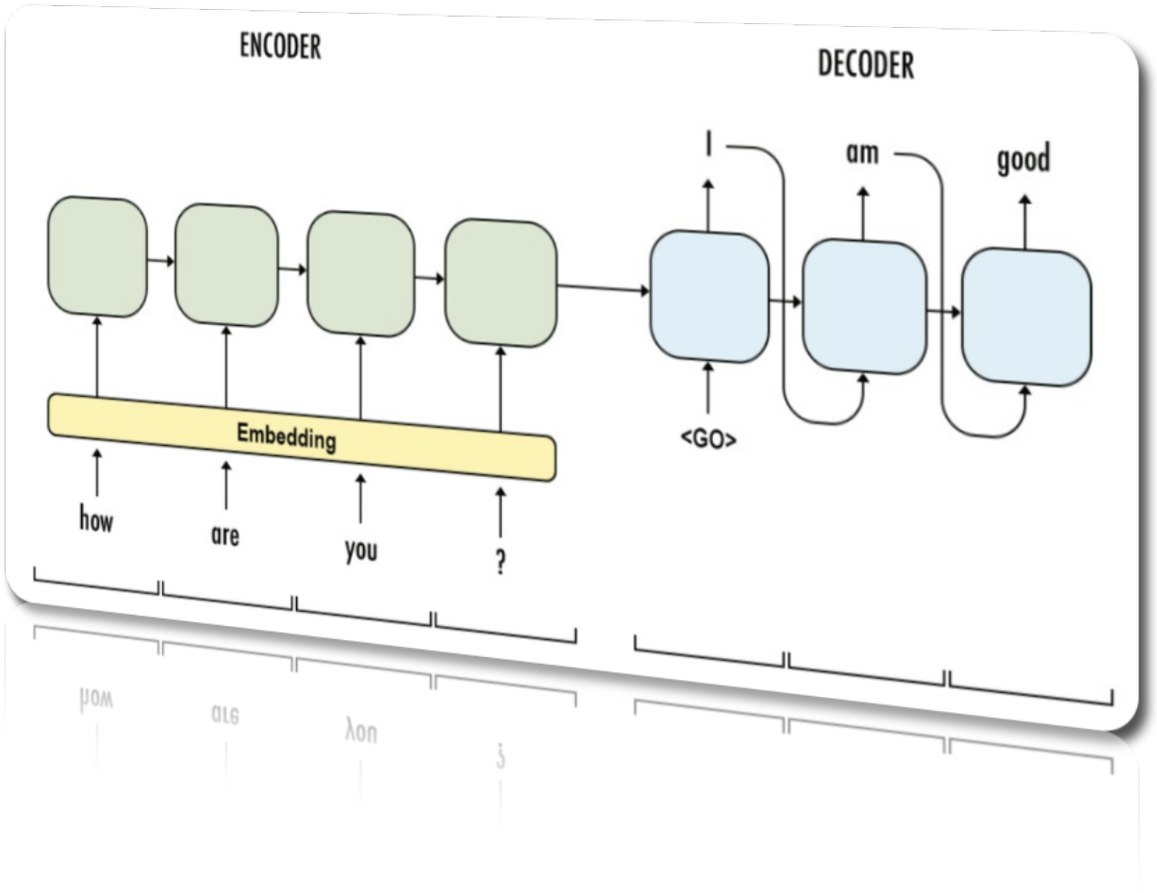
Jean Hennebert
Martin Melchior

Overview

1. Seq2Seq models

2. Seq2Seq with attention

1. Seq2Seq Models

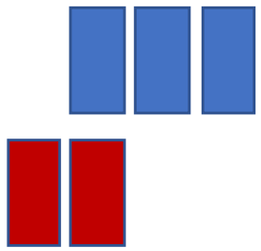


Sequence to sequence model - Seq2Seq

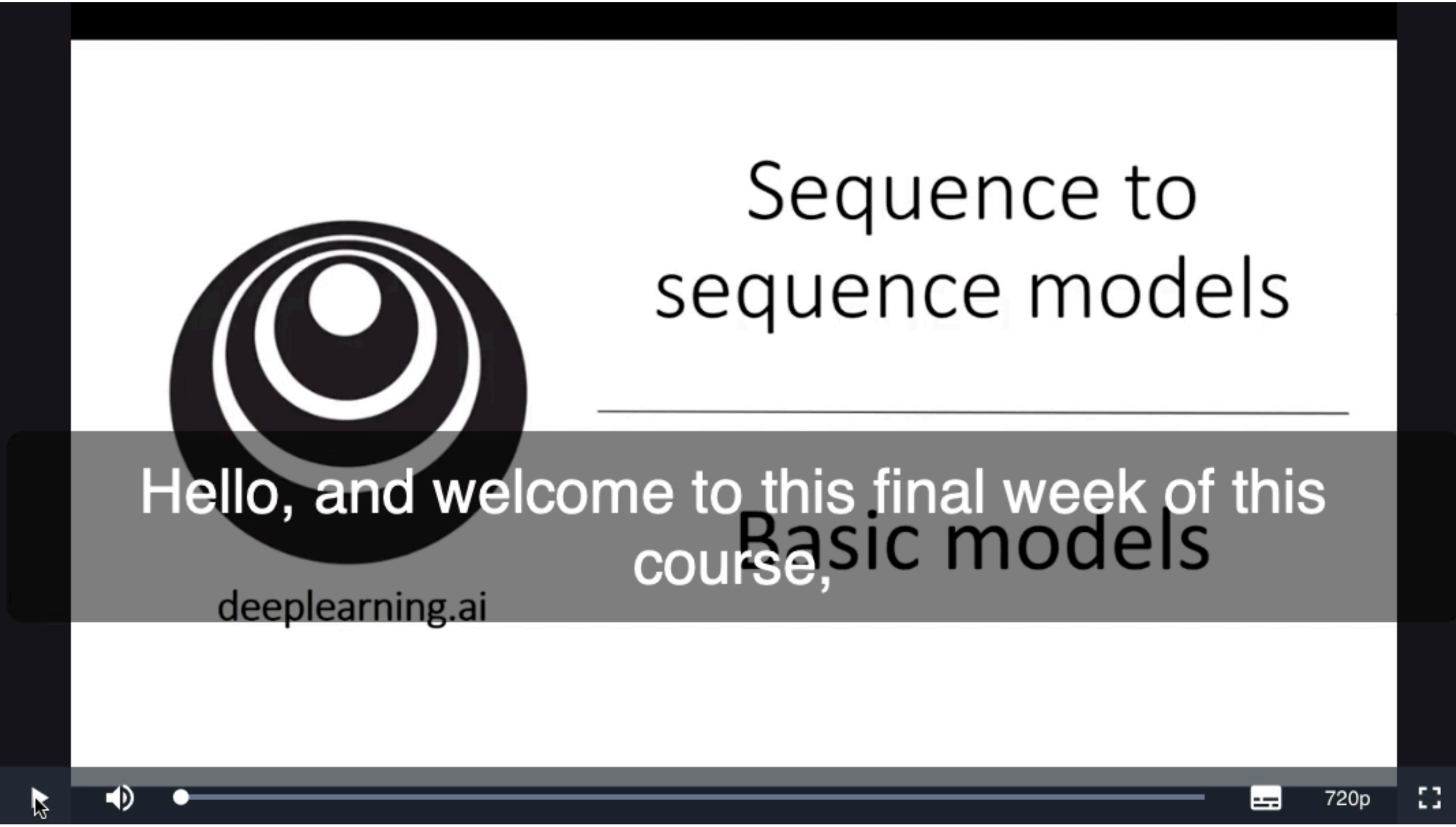
A **sequence to sequence** model is learning to map sequences of different length in situations where the entire input sequence is required in order to start predicting the target sequence.

- Typical applications are machine translation, question-answering bots, grapheme to phoneme mappers, etc
- 2 RNNs are used in an encoder-decoder architecture

many-to-many



$$T_x \geq 1, T_y \geq 1$$
$$T_x \neq T_y$$



https://youtu.be/_i3aqgKVNQI

Encoder - Decoder principle

- An **encoder** processes the input sequence and compresses the information into a context vector h_t of a fixed length
  - A “numerical summary”, also known as sentence embedding or “thought” vector.
  - This representation is expected to be a good summary of the meaning of the whole source sequence.
- A **decoder** is initialized with the context vector to emit the transformed output.
  - In its original form, Seq2Seq only used the last state of the encoder network as the decoder initial state.
  - The generation is done in a similar way as for generative model (see previous class)

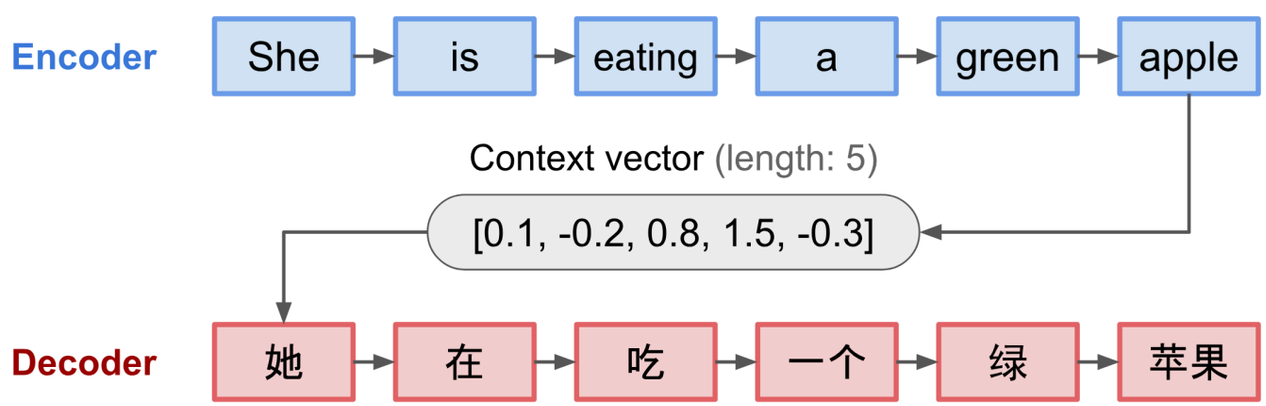


Image borrowed from: https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

Typical encoder - decoder architecture

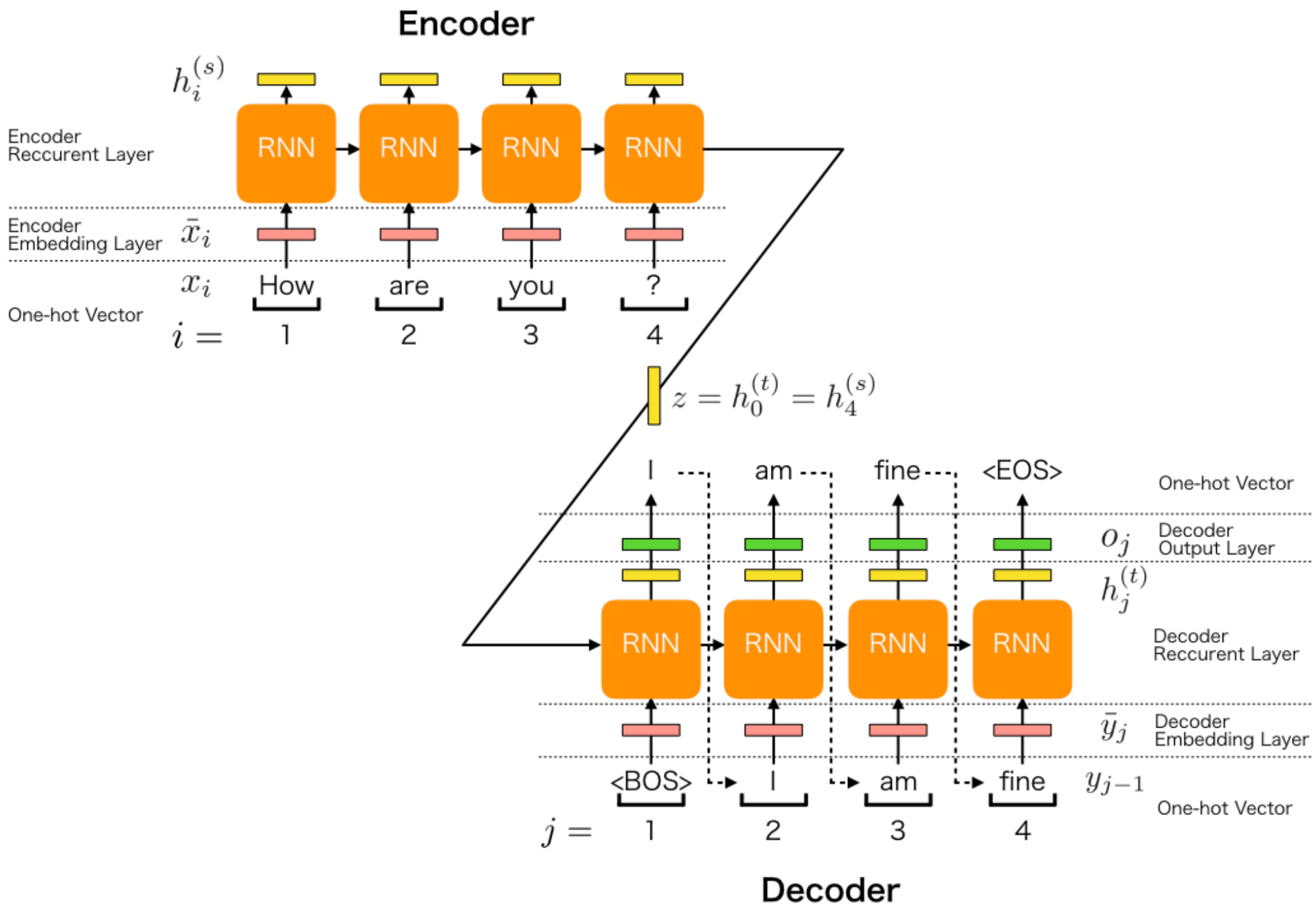


Image borrowed from: https://docs.chainer.org/en/stable/examples/seq2seq.html

Seq2Seq example in Keras

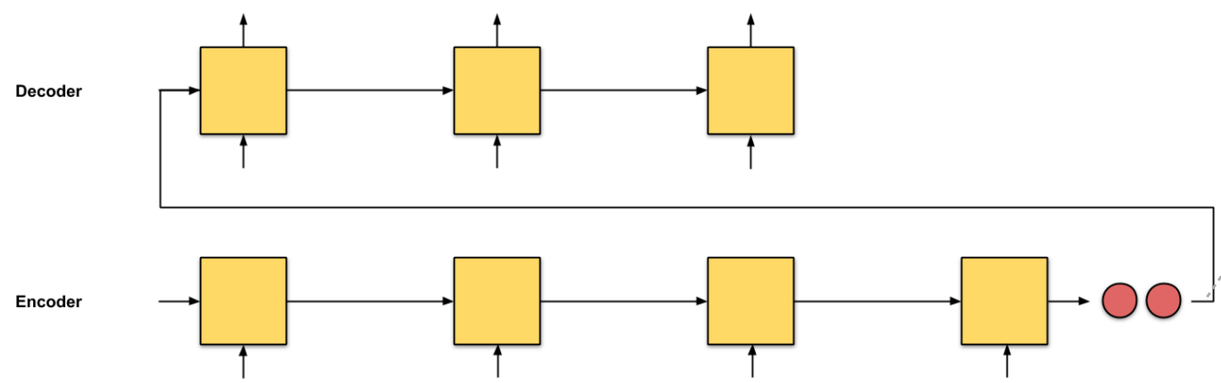
- Basic character-level Seq2Seq model.
  - Applied to translating short English sentences into short French sentences, character-by-character.
  - Note that it is fairly unusual to do character-level machine translation, as word-level models are more common in this domain.

```
Input sentence: Be nice.
Decoded sentence: Soyez
gentil !
-
Input sentence: Drop it!
Decoded sentence: Laissez
tomber !
-
Input sentence: Get out!
Decoded sentence: Sortez !
```

- Explanations
  - https://blog.keras.io/a-ten-minute-introduction-to-sequence-to-sequence-learning-in-keras.html
- Source code
  - https://github.com/keras-team/keras/blob/master/examples/lstm_seq2seq.py

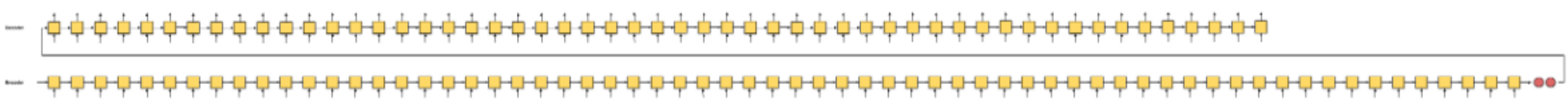
Problem of basic Seq2Seq model

- Mapping is ok for short sequences



Last encoder hidden state

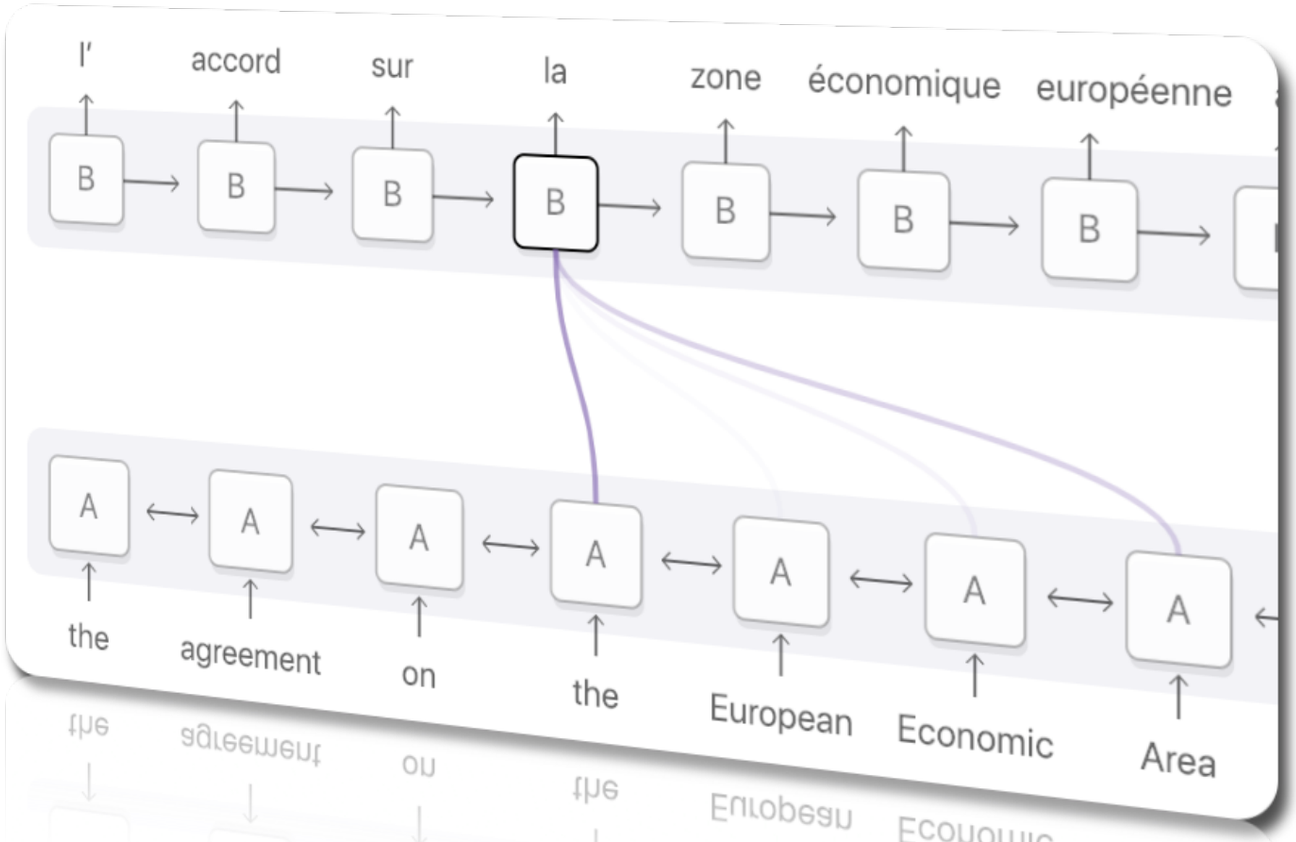
- Mapping is more difficult for long sequences
  - The context vector needs to “encode” a lot of information



- A solution is to plug-in **attention** mechanisms

Images borrowed from: https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

2. Seq2Seq with Attention



Attention

Attention is an interface between the encoder and decoder that provides the decoder with information from all (or parts of) the encoder hidden states.

- The encoder, without attention, receives only the hidden state h_t (in red below).

Two types of attention. **Global**: uses all the encoder hidden states. **Local**: uses only a subset of the encoder hidden states.

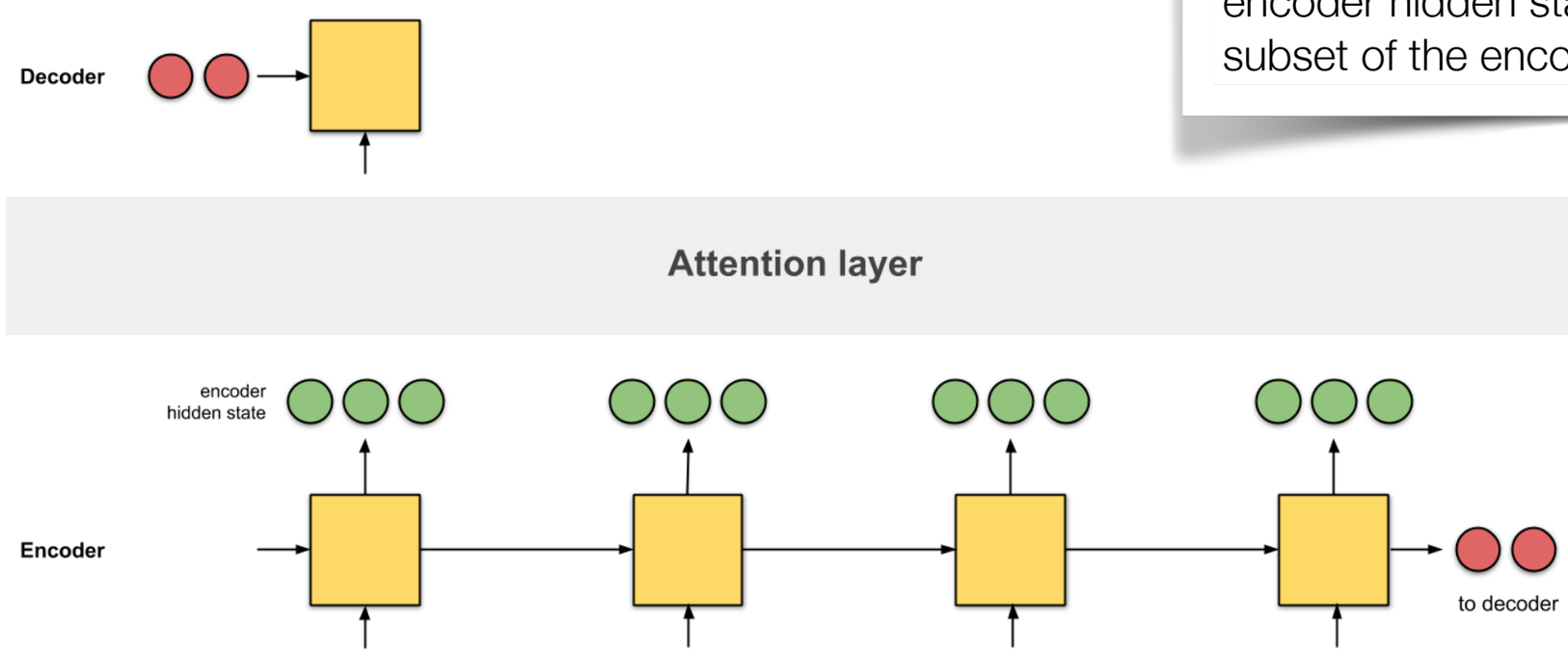


Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - intuition

- We want to provide the decoder with a notion of **alignment**
  - Which segments of original sequence match with their corresponding segments of the translation.
  - Where should the decoder “brings its attention” to map better the current output

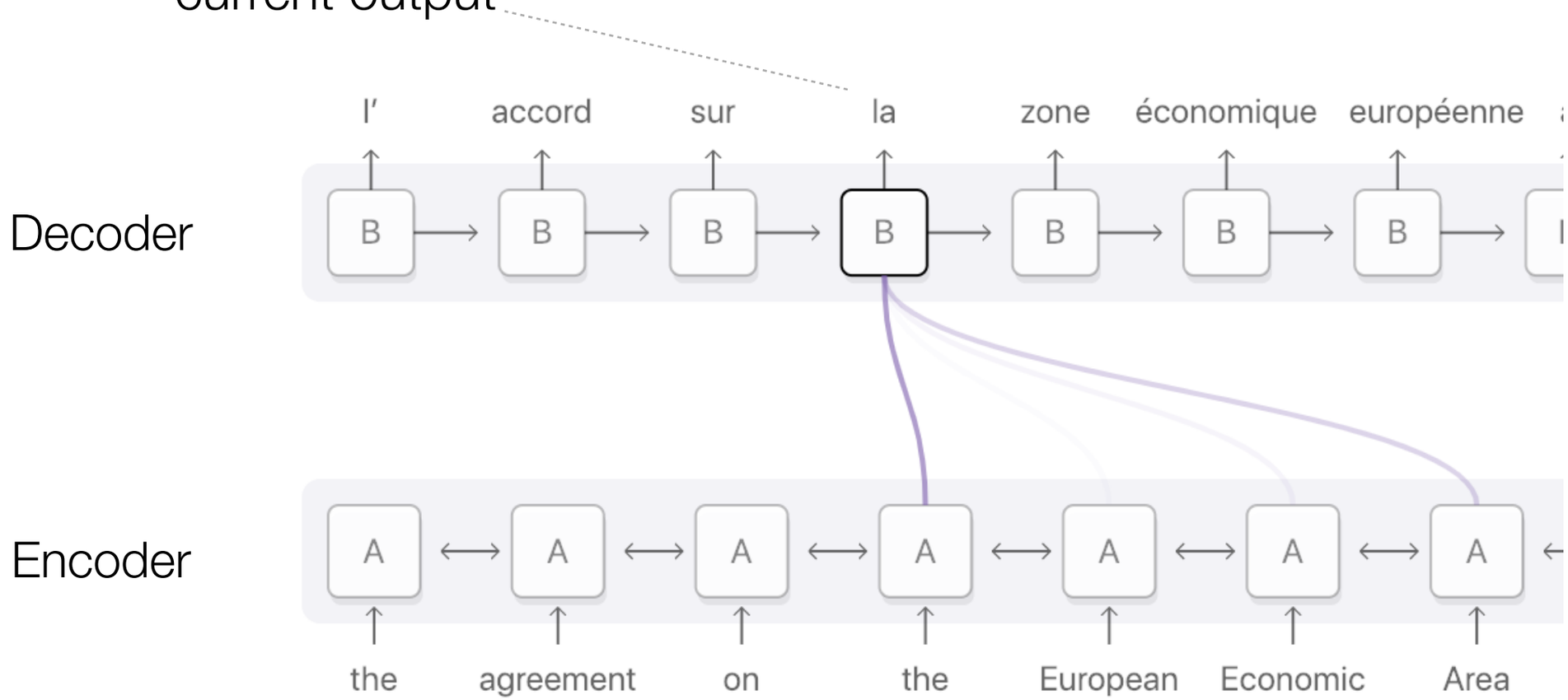


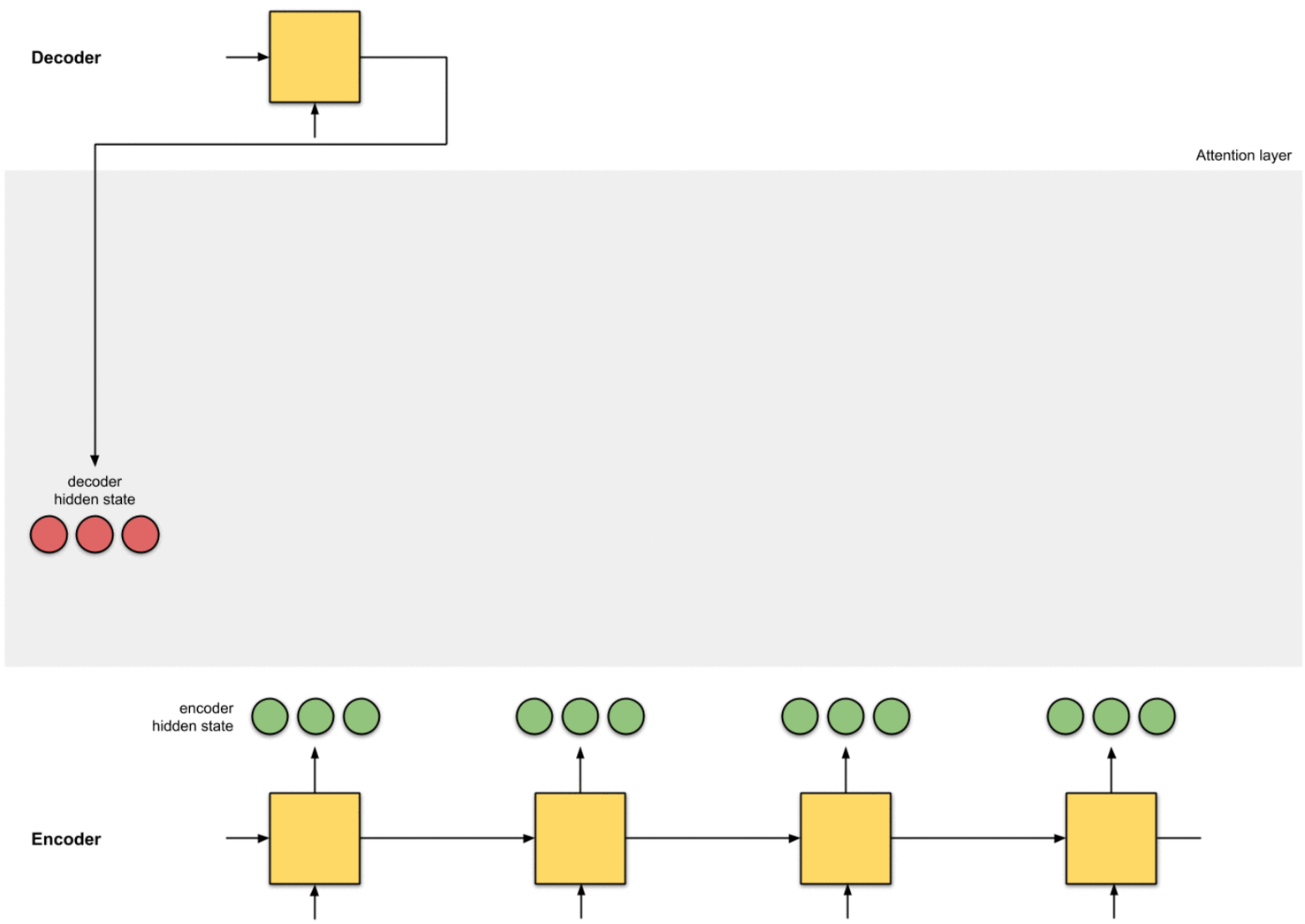
Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - intuition

- **Seq2Seq**: A translator reads the English text from start till the end. Once done, he starts translating to French word by word. It is possible that if the sentence is extremely long, he might have forgotten what he has read in the earlier parts of the text.
- **Seq2Seq with attention**: A translator reads the English text *while writing down the keywords* from the start till the end, after which he starts translating to French. While translating each English word, he makes use of the keywords he has written down.

Explanation borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

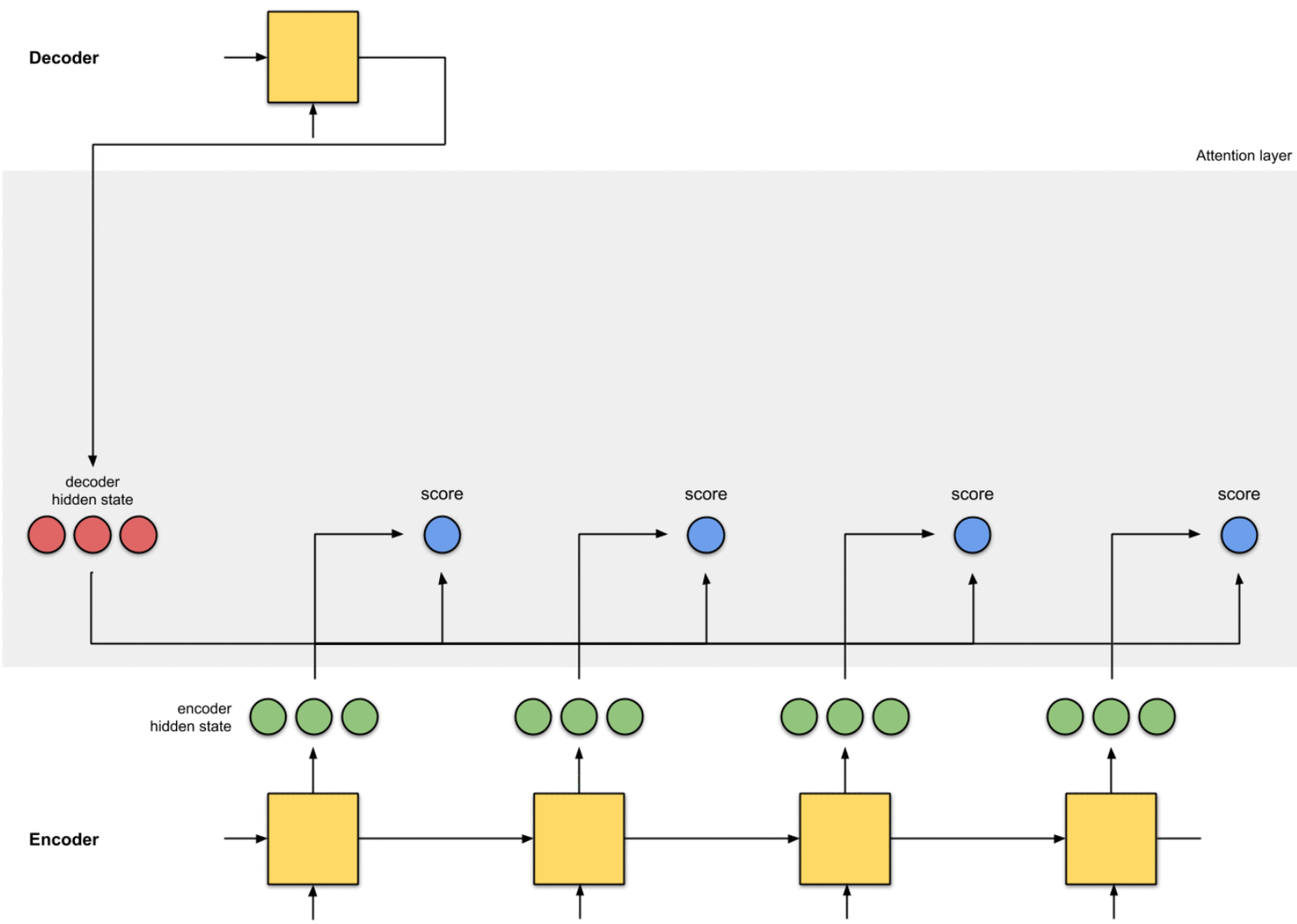
Attention - step 0: prepare hidden states



Prepare all the available encoder hidden states (green) and the first decoder hidden state (red)

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 1: obtain a score for every encoder hidden state



A **score** (scalar value) is obtained by a **score function**, aka *alignment score function* or *alignment model*.

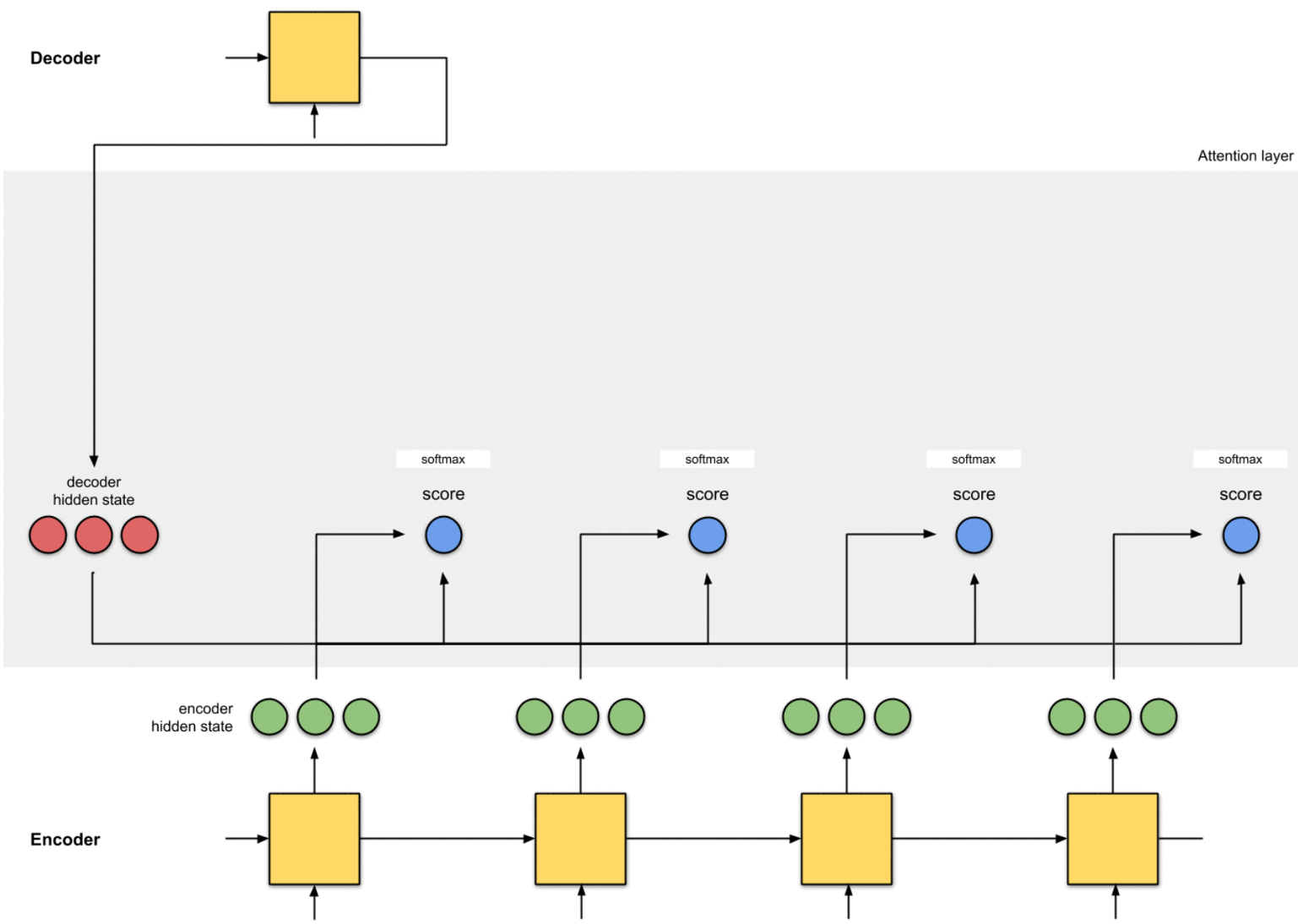
For example, the score function can be a dot product between the decoder and encoder hidden states.

```
decoder_hidden = [10, 5, 10]


encoder_hidden  score
---------------------
     [0, 1, 1]     15 (= 10×0 + 5×1 + 10×1, the dot product)
     [5, 0, 1]     60
     [1, 1, 0]     15
     [0, 5, 1]     35
```

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 2: run all the scores through a softmax layer



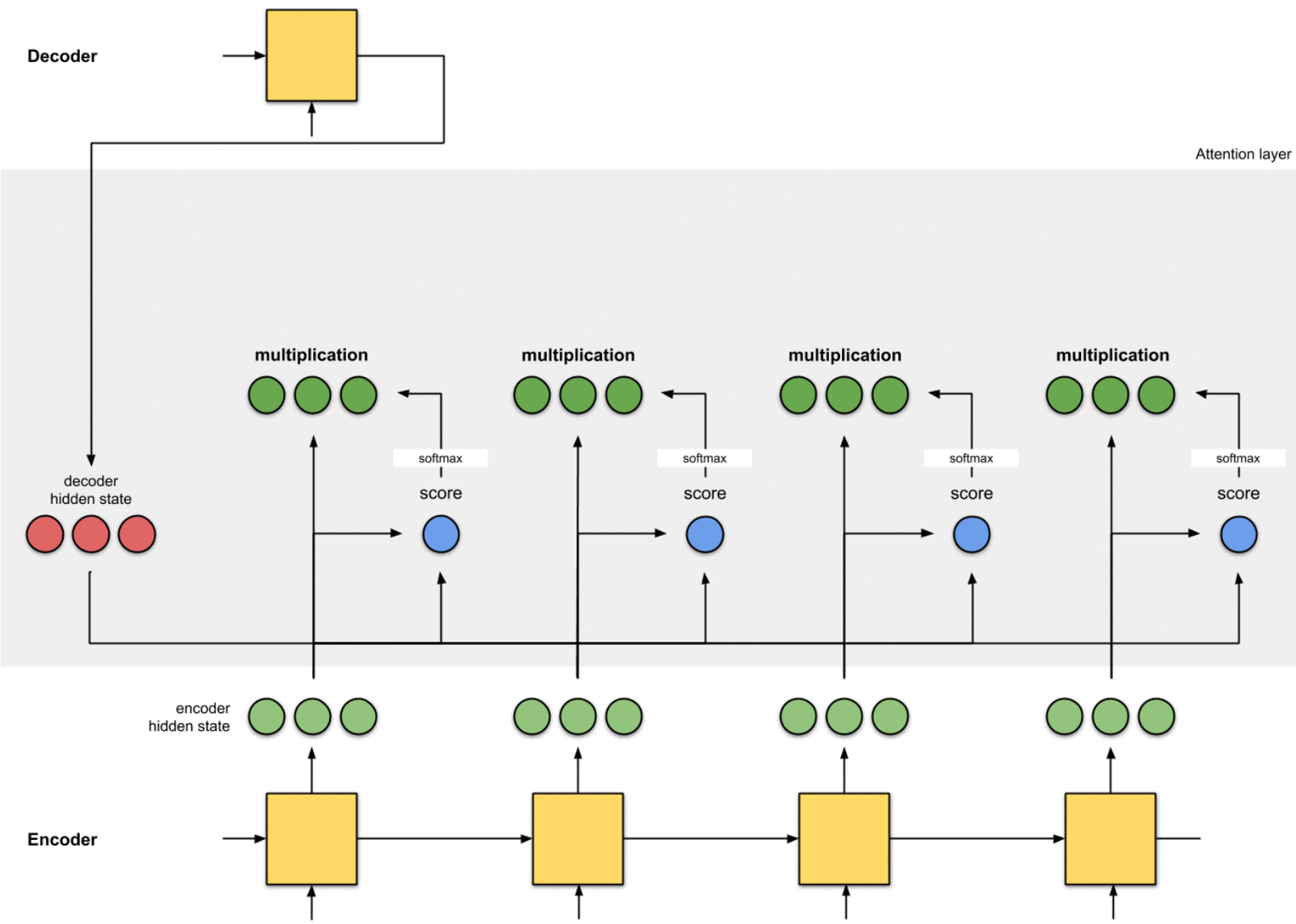
We put the scores to a softmax layer so that the softmaxed scores (scalar) add up to 1.

These softmaxed scores represent the *attention distribution.*

encoder_hidden	score	score^
[0, 1, 1]	15	0
[5, 0, 1]	60	1
[1, 1, 0]	15	0
[0, 5, 1]	35	0

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 3: Multiply each encoder hidden state by its softmaxed score



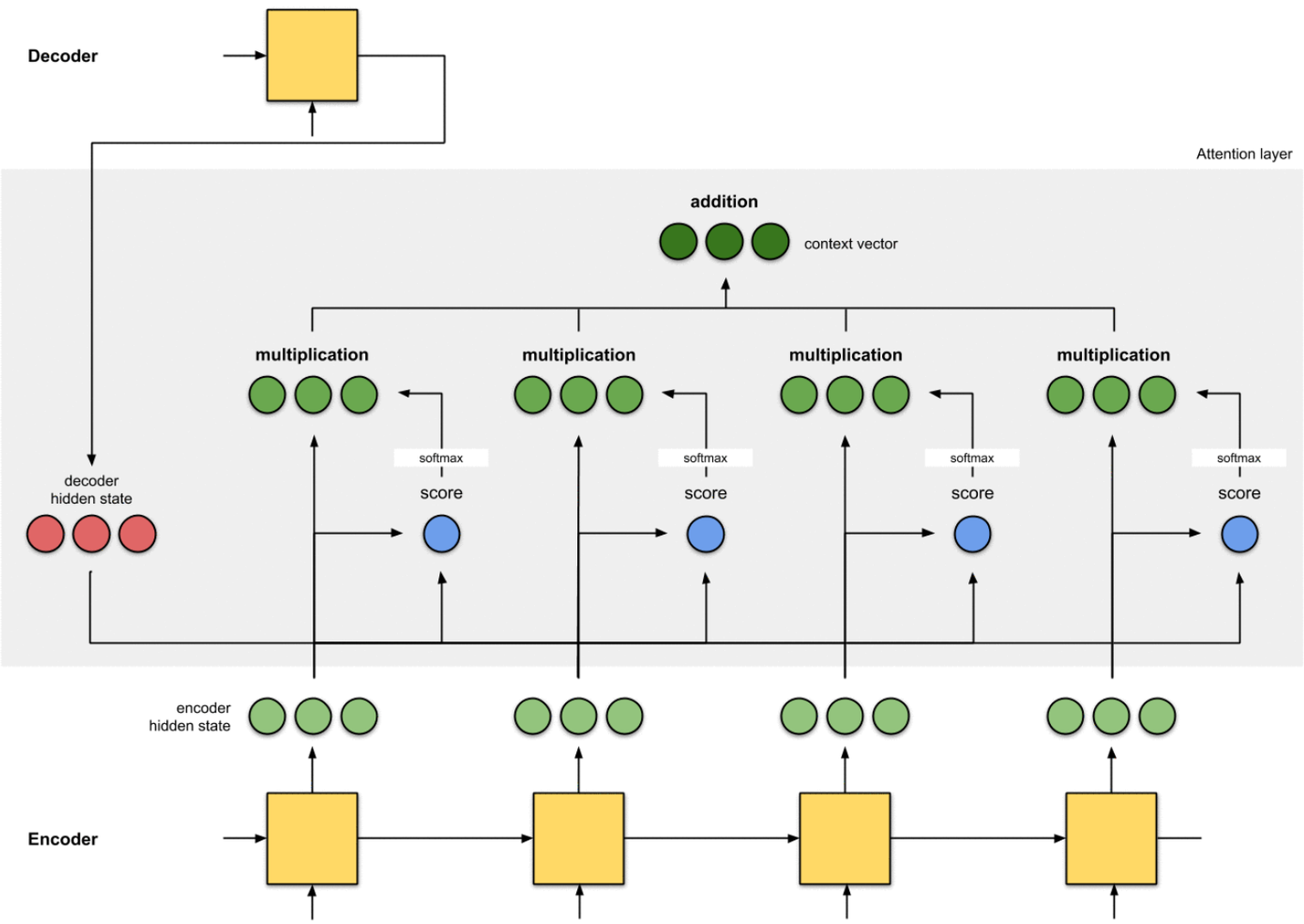
By multiplying each encoder hidden state with its softmaxed score (scalar), we obtain the *alignment vector*.

This is exactly the mechanism where alignment takes place.

encoder	score	score^	alignment
[0, 1, 1]	15	0	[0, 0, 0]
[5, 0, 1]	60	1	[5, 0, 1]
[1, 1, 0]	15	0	[0, 0, 0]
[0, 5, 1]	35	0	[0, 0, 0]

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 4: Sum up the alignment vectors.



The alignment vectors are summed up to produce the *context vector*.

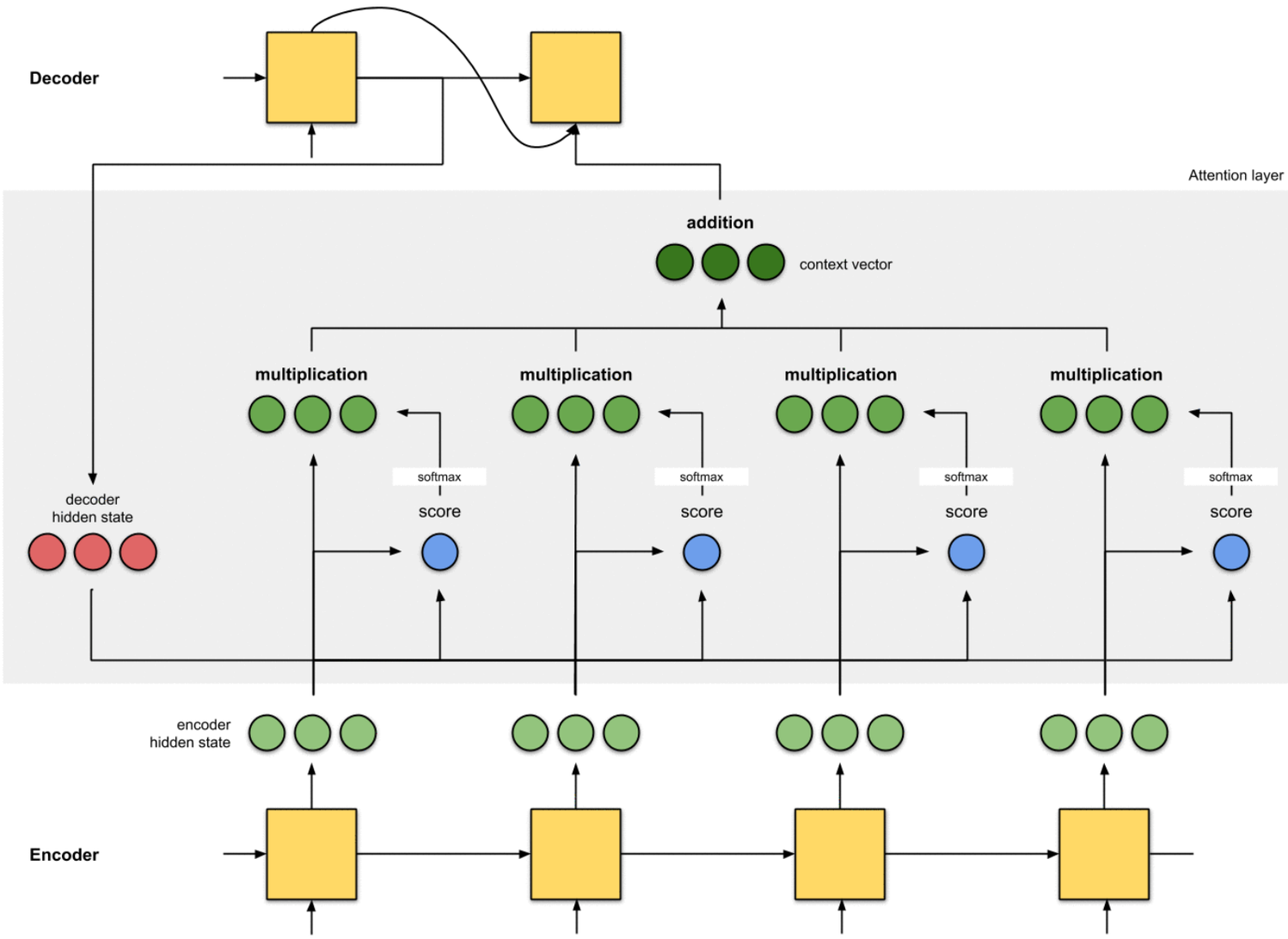
A context vector is an aggregated information of the alignment vectors from the previous step.

```
encoder  score  score^  alignment
--------------------------------
[0, 1, 1]   15     0   [0, 0, 0]
[5, 0, 1]   60     1   [5, 0, 1]
[1, 1, 0]   15     0   [0, 0, 0]
[0, 5, 1]   35     0   [0, 0, 0]


context = [0+5+0+0, 0+0+0+0, 0+1+0+0] = [5, 0, 1]
```

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 5: Feed the context vector into the decoder



The manner this is done depends on the architecture design.

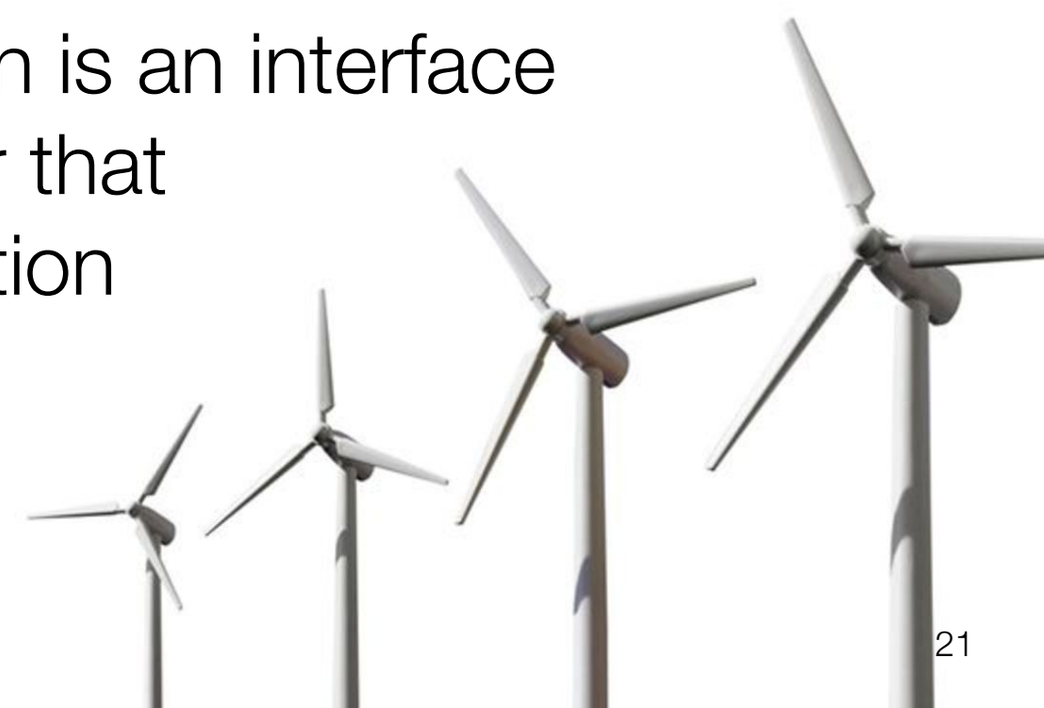
- Bahdanau et. al (2015) = concatenation of hidden state and context vector

At the next step, the decoder hidden state is used to recompute the next context vector

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - further notes

- Backpropagation will do whatever it takes to ensure that the outputs will be close to the ground truth. This is done by altering the weights in the RNNs and in the score function, if any. These weights will affect the encoder hidden states and decoder hidden states, which in turn affect the attention scores.
- In step 1, different score functions can be tried: (i) additive/concat, (ii) dot product, (iii) location-based, and (iv) 'general'
- Alternate architectures have been built on top of what is explained before: https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Wrap-up

- A **Seq2Seq** model is learning to map sequences of different length in situations where the entire input sequence is required in order to start predicting the target sequence.
  - Encoder - Decoder architecture with 2 RNNs
  - h_t is passed from the last step of the encoder as input to the decoder
- In **Seq2Seq + attention,** attention is an interface between the encoder and decoder that provides the decoder with information from all (or parts of) the encoder hidden states.